

AI Agentic Development

Petr Wehrenberg

"Vibe coding is a vibe. Shipping is a discipline."

Agenda

1. How AI actually works — tokens, attention, hallucinations
2. The AI coding landscape — competitors & Claude Code
3. Vibe coding vs. agentic coding
4. Claude Code deep dive — skills, hooks, agents, MCP
5. Specs are back — a brief history

How AI Actually Works

Before you use a tool, understand what it is

What is a token?

LLMs don't read words — they read **tokens**.

```
"unhelpful" → ["un", "help", "ful"]  
"useState" → ["use", "State"]  
"<button>" → ["<", "button", ">"]
```

- A token is roughly $\frac{3}{4}$ of a word on average
- ~750 words $\approx$ 1000 tokens
- Numbers, symbols, and rare words split into more tokens
- Code tends to tokenize less efficiently than prose

Why it matters: you pay per token, and the model has a limit.

The context window = working memory

Context Window (e.g. 200,000 tokens)

CLAUDE.md | your files | chat history | 

↑ you are here

- Everything the model can "see" at once lives in the context window
- When context is full, **earlier content is forgotten**
- This is why CLAUDE.md at the **top of context** matters
- Bigger context window = more files, more history, better decisions

Embeddings — meaning as coordinates

Words and code are converted into **vectors** in high-dimensional space.

```
"function" → [0.12, -0.87, 0.45, ...]  
"method"   → [0.11, -0.84, 0.47, ...] ← close!  
"variable" → [0.55,  0.21, -0.33, ...] ← further
```

- Similar concepts live **close together** in vector space
- The model understands synonyms, patterns, and relationships
- Enables semantic search, retrieval, and code understanding

Attention — reasoning across long files

Every token **attends to** every other token.

```
function calculateTotal(items) {      ← defined here
  return items.reduce(...)
}

// 300 lines later...

const result = calculateTotal(cart) ← model connects these
```

- The model tracks long-range dependencies across the whole file
- More layers = more complex relationships tracked
- This is why pasting full files gives better results than snippets

Why AI hallucinates

LLMs predict the **most probable next token** — they don't "know" facts.

```
// Model generates this confidently:  
import { useSomething } from '@tanstack/react-query'  
// useSomething doesn't exist. It looks plausible.
```

- No strong signal in training data → confident wrong answer
- The model has never "run" code — it pattern-matches
- **API names, library versions, function signatures** are common failure points

"The model is always trying to be helpful. That's the problem."

Prompts shape output

Vague prompt → vague output. Context-rich prompt → precise output.

Prompt	Output quality
<code>"add a button"</code>	Generic HTML or whatever it guesses
<code>"add a submit button using our Shadcn Button component with variant='default'"</code>	Correct code using project patterns
<code>"add a submit button"</code> + CLAUDE.md with stack details	Correct code without spelling it out

The better your context, the less you have to say.

Checkpoint: How AI Works

- Tokens = chunks of text, $\sim\frac{3}{4}$ of a word on average
- Context window = working memory, has a hard limit
- Embeddings = meaning encoded as vectors (similar = close)
- Attention = model reasons across the whole context
- Hallucinations happen because the model pattern-matches, not reasons
- Prompts + context files = the quality multiplier

Questions?

The AI Coding Landscape

Know what you're choosing and why

AI coding tools in 2026

Tool	Description
GitHub Copilot	IDE inline autocomplete with recently added Agent Mode
Cursor	VS Code fork, deep AI integration, huge community
Windsurf	IDE with fully agentic Cascade multi-step assistant
Antigravity	Google's agent-first IDE with built-in browser control
Codex	OpenAI's cloud agent, runs sandboxed, pushes to GitHub
Kiro	Amazon's spec-first IDE, generates design docs before code
Claude Code	Terminal-native agentic CLI, skills, hooks, MCP, subagents

Why Claude Code?

It's the only tool built **agentic-first from day one**.

- **Terminal-native** — lives in your workflow, not around it
- **Skills** — reusable, shareable, version-controlled prompt workflows
- **Hooks** — automate actions before/after every AI operation
- **MCP** — connect AI to any tool: browser, database, Jira, Figma
- **Subagents** — delegate subtasks to parallel worker agents
- **Marketplace** — marketplace for distributing and installing skills

Other tools added agentic features. Claude Code was designed for it.

Checkpoint: The Landscape

- Tools range from autocomplete (Copilot) to spec-first IDEs (Kiro) to cloud agents (Codex)
- Claude Code differentiates on: extensibility, MCP, skills, hooks
- No tool is perfect — choose what fits your workflow
- For this course: Claude Code is the focus, principles transfer to others

Questions?

Vibe Coding vs. Agentic Coding

One is a feeling. The other is a practice.

What is vibe coding?

A term coined by Andrej Karpathy in 2025.

```
You: "make it look nicer"  
AI:  [generates 200 lines]  
You: "looks good, ship it"
```

- Prompt → accept → ship, with minimal understanding
- You don't read the code. You feel whether it works.
- Fast feedback loop, low cognitive load

For prototypes and throwaway projects: totally fine.

The vibe coding trap

The problem isn't the first feature. It's the fifth.

```
Week 1: app works, vibes are immaculate ✓  
Week 3: "why is there a useEffect here?"  
Week 5: adding a button breaks auth somehow  
Week 8: nobody knows how anything works
```

- No mental model → bugs are mysterious
- Technical debt accumulates invisibly
- You lose **ownership** of your own codebase
- The AI is the only one who understands it — and it forgets every session

Vibe coding doesn't scale. It compounds.

What is agentic coding?

You are still the engineer. The AI is the power tool.

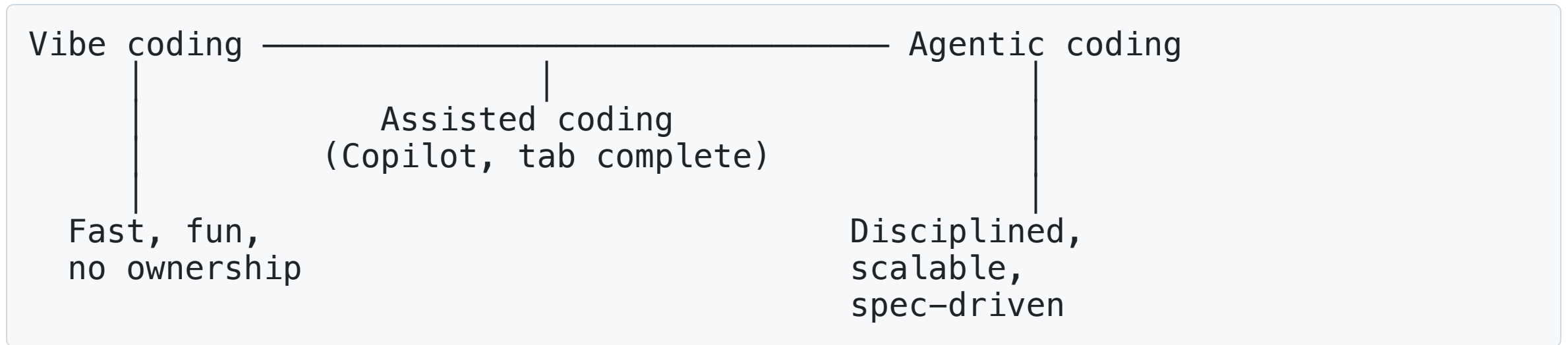
```
You: [write spec] "implement user authentication:  
  - JWT tokens, 24h expiry  
  - refresh token rotation  
  - use our existing Drizzle schema"
```

```
AI: [implements across 6 files, writes tests]
```

```
You: [review, evaluate, accept or reject]
```

- You define **what** and **why** — AI handles **how** and **where**
- You evaluate every output against your mental model
- You own the spec, which means you own the outcome

The spectrum



Most students start somewhere in the middle. That's fine.

Goal: move right as the project grows.

Checkpoint: Vibe vs. Agentic

- Vibe coding = prompt → accept → ship, minimal understanding
- Fine for prototypes, dangerous for production
- Debt accumulates invisibly until the codebase is unownable
- Agentic coding: you own the spec, AI handles execution
- You evaluate every output — always

Questions?

Claude Code Deep Dive

The toolkit

CLAUDE.md — your AI's manual

The file Claude reads **before doing anything** in your project.

Project: PB138 Monorepo

Stack

- Frontend: React 19, Vite, TypeScript, Tailwind, Shadcn
- Backend: Express, Drizzle ORM, PostgreSQL
- API client: generated Kubb client (never use fetch directly)

Patterns

- Data fetching: React Query hooks from @verdice-client
- Forms: React Hook Form + zod validation
- No `any` types – use proper interfaces

Commands

- `npm run dev` – start dev server
- `npm run typecheck` – run TypeScript

Skills — reusable prompt workflows

Skills are **version-controlled, shareable prompts** for common tasks.

```
/commit      – writes a conventional commit message
/review      – pragmatic code review against project standards
/feature-dev – guided feature development workflow
/tdd         – test-driven development loop
```

- Each skill is a **folder** with a `SKILL.md` file inside `.claude/skills/`
- Can be shared across teams and projects
- Encode your team's best practices into repeatable workflows

```
# invoking a skill
> /commit
> /review
> /feature-dev add user profile page
```

Skills — file structure

```
.claude/  
└─ skills/  
    └─ commit/  
        └─ SKILL.md  
    └─ feature-dev/  
        └─ SKILL.md  
        └─ examples.md    ← optional supporting files
```

Each `SKILL.md` has a frontmatter header + instructions:

```
---  
name: commit  
description: Write a conventional commit message for staged changes  
---
```

Look at the staged changes with ``git diff --staged``. Write a single-line...

Hooks — automation at every step

Hooks run **shell commands** before or after Claude's tool calls.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [{ "type": "command", "command": "npm run lint --fix" }]
      }
    ]
  }
}
```

- Run tests before accepting a file edit
- Auto-format after every change
- Log all AI actions for audit trails

Permissions — safe but not annoying

Three levels of settings files, applied in order:

<code>~/.claude/settings.json</code>	← user-level (all your projects)
<code>.claude/settings.json</code>	← project-level (committed, shared with team)
<code>.claude/settings.local.json</code>	← project-local (gitignored, personal overrides)

The goal: **Claude doesn't interrupt you for safe operations**, but still asks for anything risky.

Permissions — allowedTools & deniedTools

```
// .claude/settings.json (project-level, commit this)
{
  "permissions": {
    "allow": [
      "Edit",
      "Read",
      "Bash(npm run *)",
      "Bash(git status)",
      "Bash(git log *)"
    ],
    "deny": ["Bash(git push --force*)"]
  }
}
```

- `Bash(npm run *)` — allow any npm script, no confirmation
- `Bash(git push --force*)` — blocked entirely, always
- Anything not listed → Claude asks before running

MCP — Model Context Protocol

MCP connects Claude to **external tools and data sources**.

Claude Code



mcp:playwright	→ controls a browser
mcp:github	→ reads PRs, creates issues
mcp:atlassian	→ reads Jira tickets
mcp:figma	→ reads design files
mcp:filesystem	→ reads/writes your local files

- Standardized protocol — any tool can expose an MCP server
- Claude can **use** these tools autonomously during a task

MCP — Hello World

Add an MCP server in `.mcp.json` at the root of your project:

```
{
  "mcpServers": {
    "playwright": { "command": "npx", "args": ["@playwright/mcp@latest"] }
  }
}
```

Claude now has access to browser tools — no extra code needed:

<code>browser_navigate(url)</code>	→ open a page
<code>browser_click(selector)</code>	→ click an element
<code>browser_fill_form(...)</code>	→ fill inputs
<code>browser_take_screenshot()</code>	→ capture the page
<code>browser_snapshot()</code>	→ accessibility tree (what Claude "sees")

Subagents — divide and conquer

Claude can **spawn subagents** to work on independent tasks in parallel.

```
Main agent: "implement the user dashboard"  
├── Subagent A: write the API endpoints  
├── Subagent B: write the React components  
└── Subagent C: write the tests
```

- Each subagent has its own context window
- They work independently, then report back
- The main agent integrates the results
- Massively reduces time on large features

Subagents vs. Parallel agents

Two different ways to scale — often used together.

	Subagents	Parallel agents
Who starts them	Claude (automatically)	You (manually)
How	Agent tool call in a prompt	Separate terminal panes / tmux
Coordination	Main agent integrates results	You coordinate
Context	Shared session, isolated window	Fully independent processes
Best for	Breaking one task into parts	Running completely separate tasks

Spec-driven development

Write the spec first. Let the agent implement it.

Feature: User Notifications

Requirements

- Users receive in-app notifications for new comments
- Mark as read individually or all at once
- Persist notification state in the database

Acceptance Criteria

- [] GET /api/notifications returns unread count
- [] UI shows badge count in the navbar

Tools: spec-kit, specify — good for understanding the approach.

For your project: use **superpowers** — it's less token-expensive and well-suited for you

Superpowers — the ecosystem

Superpowers is a plugin in **marketplace** for Claude Code.

```
superpowers/  
├── brainstorming      – turn ideas into specs through dialogue  
├── writing-plans       – create implementation plans  
├── tdd                – test-driven development loop  
├── debugging          – systematic debugging workflow  
├── code-reviewer      – review against project standards  
├── finishing-a-branch – wrap up, PR, cleanup  
└── ...20+ more skills
```

- Install once, use across all your projects
- Encodes senior engineering practices into repeatable prompts
- Open source — you can read, fork, and extend any skill

Checkpoint: Claude Code Toolkit

- **CLAUDE.md** — project context, read before every session
- **Skills** — reusable, shareable workflows for common tasks
- **Hooks** — shell automation around every AI action
- **Permissions** — three levels (user / project / local), allow safe ops, block dangerous ones
- **MCP** — connect Claude to browsers, databases, external tools
- **Subagents** — parallel workers for independent subtasks
- **Spec-driven dev** — write the spec, let AI implement
- **Superpowers** — community skill ecosystem

Questions?

Specs Are Back

A brief history of where we went wrong — and right again

Era 1: The age of specs (1970s–1990s)

Before agile, before GitHub, before the internet was fast.

- **Waterfall:** requirements → design → implementation → testing
- Specs were 200-page PDF documents written before a line of code
- UML diagrams for every class, sequence diagram for every interaction
- **The good:** clear contracts, shared understanding
- **The bad:** reality didn't match the spec by delivery day

"Plans are useless. Planning is indispensable." — Eisenhower

Era 2: Agile kills the spec (2000s–2010s)

The Agile Manifesto, 2001:

"Working software over comprehensive documentation"

- Specs were waste — they became outdated the moment they were written
- Move fast, iterate, the code **is** the documentation
- User stories replaced blueprints
- **The good:** faster delivery, closer to reality
- **The bad:** the institutional memory lived only in developers' heads

The entropy problem

Code accumulates **entropy** faster than it accumulates understanding.

```
Year 1: codebase is small, everyone knows it  
Year 2: "why is this here?" – first mystery  
Year 3: "don't touch that" – first forbidden zone  
Year 5: entire subsystems that only one person understands  
Year 7: rewrite discussion begins
```

- The codebase is the most complete truth — but it's also the hardest to read
- Onboarding new engineers takes months, not days
- AI tools make this **worse**: they generate code faster than humans can understand it

AI resurrects the spec

The pendulum is swinging back — for a new reason.

```
CLAUDE.md      → project blueprint for AI  
Skills         → encoded workflows and standards  
Specs          → what to build, before building it  
Architecture docs → why things are the way they are
```

- AI needs context to produce good output
- That context **is** a spec
- Writing a good CLAUDE.md forces you to articulate your architecture
- Writing a good skill encodes your team's best practices

Every good AI setup is secretly a well-documented project.

The new spec layer

Specs aren't dead. They just look different now.

Old world	New world
200-page PDF	CLAUDE.md (2 pages)
UML sequence diagrams	Skill workflows
Architecture Decision Records	Agent-readable blueprints
Jira tickets	Spec files fed to agents
Onboarding docs	CLAUDE.md + skills repo

The difference: these specs are **machine-readable and action-oriented**.

"We thought specs were dead. AI just brought them back with a different interface."

Wrapping Up

Key takeaways

1. **AI is a pattern-matcher**, not a reasoner — always evaluate its output
2. **Vibe coding** is fine for prototypes, a trap for production
3. **Agentic coding** = you own the spec, AI handles execution
4. **Claude Code** toolkit: CLAUDE.md, skills, hooks, MCP, subagents
5. **Specs are back** — not as bureaucracy, but as AI fuel

"AI makes you faster."

"Discipline makes you better."

"Specs make both sustainable."